Bilkent University Electrical and Electronics Department

EE102-01 Lab 6 Report:

Arbitrary Waveform Generator

Muhammed Mesud Bilirgen – 22301965 – 7th April 2025

Purpose

The aim of this experiment is to design and implement an arbitrary digital waveform generator using the "Clocking Wizard" IP in Vivado on the Basys3 board. The waveform follows a custom timing pattern determined by the designer. The waveform is verified through both simulation and physical testing via oscilloscope. The experiment aims to enhance understanding of time-based digital design through waveform generation using VHDL.

Design Specification

The design consists of only one module which is lab6.vhd producing a time-based pattern corresponding to Morse code representations of the words ROCKY and BALBOA. The circuit includes:

Inputs:

- clk_100MHz (clock input from Basys3 board)
- reset (manual reset input)
- mode (selects between two output waveforms: ROCKY or BALBOA)

Output: pulse_out (digital signal representing the selected Morse waveform)

The system uses a Clocking Wizard IP to generate a 10 MHz clock signal from the default 100 MHz input, so as to implement precise timing control, where each Morse time unit (T) equals 100 clock cycles (10 µs). The waveform generator supports two predefined patterns:

ROCKY: (R) . – . (O) – – – (C) – . – . (K) – . – (Y) – . – –

BALBOA: (B) – . . . (A) . – (L) . – . . (B) – . . . (O) – – – (A) . –

(Each letter is separated by a 3T LOW gap. A dot is HIGH signal for 1T, a dash is HIGH signal for 3T.)

Methodology

A Clocking Wizard IP core was used to derive a 10 MHz clock from the board's internal clock 100 MHz (Figure 1). A single process was used to increment a counter and generate the desired waveform. The signal output is altered by if-elsif conditions based on the counter value. A mode input allows selection between two different sequences.
Simulation was performed to verify the timing accuracy and logic behavior. The waveform output was also routed to a pin on the Basys3 board and observed by an oscilloscope.

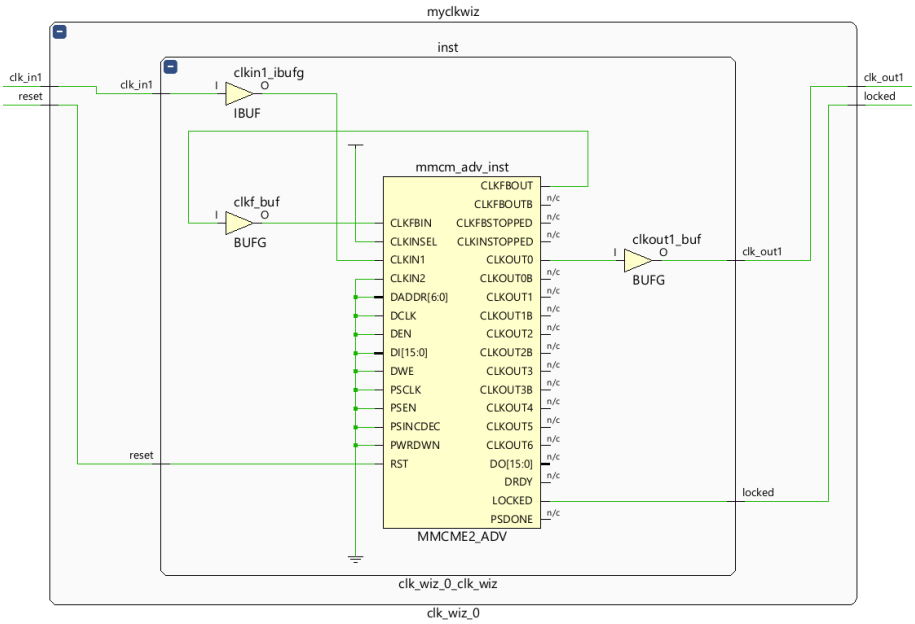

Figure 1: The clock wizard part of RTL schematic

Results

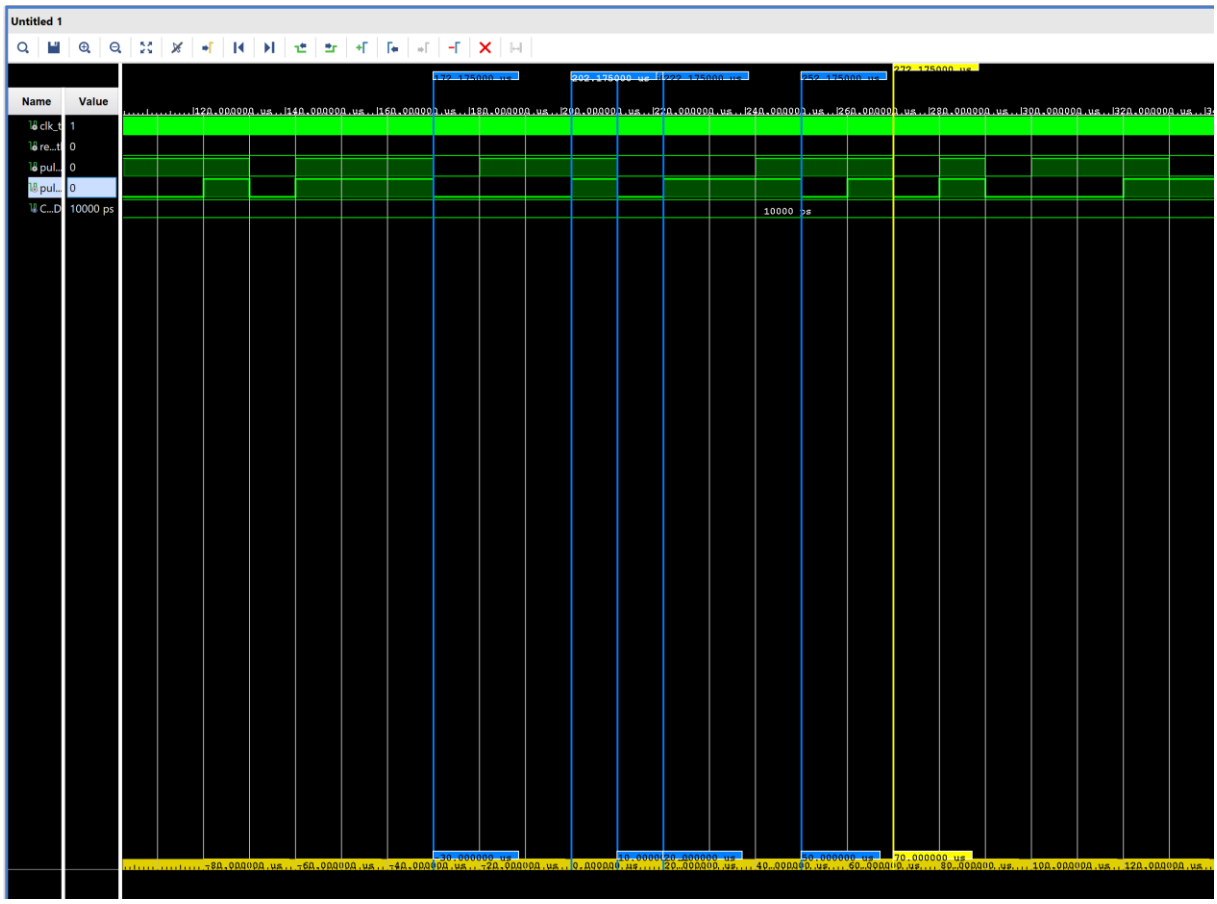

Figure 2: 30 us low, 10 us high, 10 us low and 30 us high

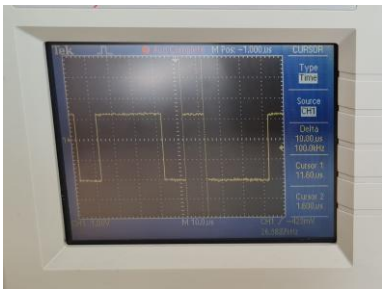
Figure 3: 10 us high

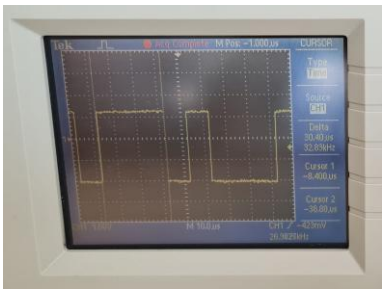
Figure 4: 30 us high

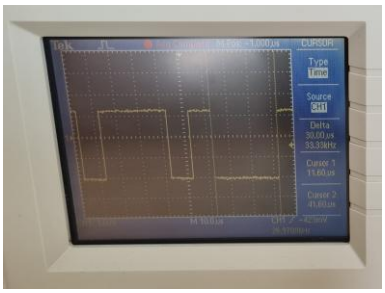
Figure 5: 30 us low

The waveform generator was verified through both simulation and physical testing. The waveform timing was measured using markers in simulation (Figure 2) and cursors in oscilloscope to ensure that:

- Dots are 10 µs (Figure 3).
- Dashes are 30 µs (Figure 4).
- Spacing between letters are 30 µs (Figure 5).

In simulation, the output waveform was observed for both modes (Figure 6). The oscilloscope clearly displayed the repeating Morse waveforms. Figure 7 shows the oscilloscope output of the ROCKY waveform. Figure 8 shows the BALBOA waveform. As a result, the measured durations of pulses matched the expected timing values.

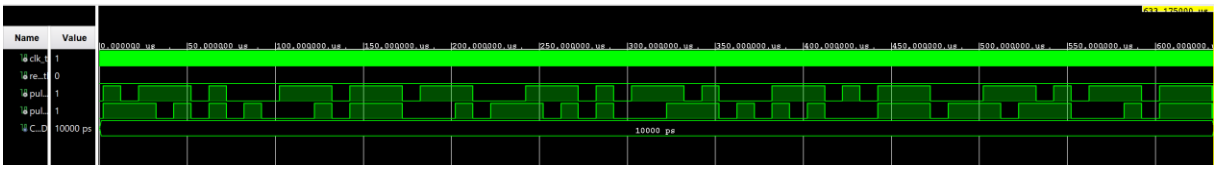

Figure 6: The waveform from simulation for mode 0 and 1

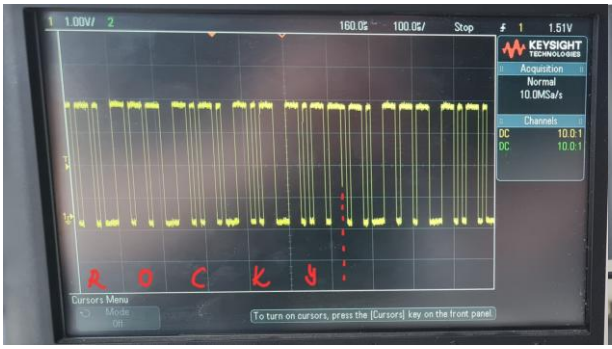

Figure 7: The waveform from oscilloscope for mode = '0'

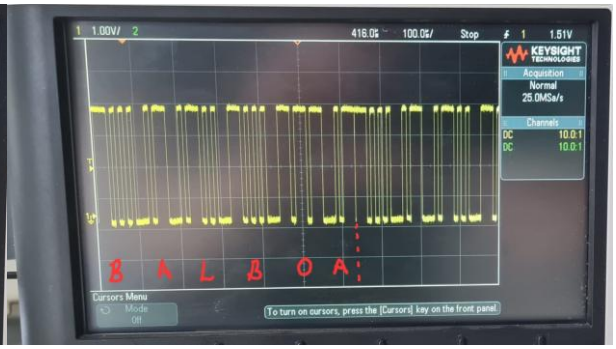

Figure 8: The waveform from oscilloscope for mode = '1'

Conclusion

The objective of this experiment was to design and implement an arbitrary digital waveform generator. The design successfully utilized the Clocking Wizard IP to generate a 10 MHz clock, enabling accurate control of pulse durations. Two Morse code sequences representing the words ROCKY and BALBOA were implemented with a selection inputs. The waveform behaviour was verified through both simulation and oscilloscope measurements. Overall, the experiment was successful, demonstrating precise timing control. This lab enhanced understanding of clock domain manipulation, signal timing, and time-based digital logic design using VHDL.

Appendices

lab6.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity lab6 is
   Port (
      clk_100MHz : in  STD_LOGIC;
      reset      : in  STD_LOGIC;
      pulse_out  : out STD_LOGIC;
      mode       : in STD_LOGIC := '0');
end lab6;

architecture Behavioral of lab6 is

   component clk_wiz_0
      Port (
         clk_in1  : in  STD_LOGIC;
         clk_out1 : out STD_LOGIC;
         reset    : in  STD_LOGIC;
         locked   : out STD_LOGIC
      );
   end component;

   signal clk_10MHz  : STD_LOGIC;
   signal locked    : STD_LOGIC;
   signal counter   : integer := 0;
   signal pulseR     : STD_LOGIC := '0';
   signal pulseB     : STD_LOGIC := '0';

begin

   myclkwiz : clk_wiz_0
      port map (
         clk_in1  => clk_100MHz,
         clk_out1 => clk_10MHz,
         reset    => reset,
         locked   => locked
      );

   process(clk_10MHz)
   begin
      if rising_edge(clk_10MHz) then
         if reset = '1' or locked = '0' then
            counter <= 0;
            pulseR <= '0';
         else -- (dot = 100, dash = 300, space = 100, letter gap = 300)
            -- ROCKY
            -- R:
            if counter < 100 then          -- dot
               pulseR <= '1';
            elsif counter < 200 then
               pulseR <= '0';
            elsif counter < 500 then       -- dash
               pulseR <= '1';
            elsif counter < 600 then
               pulseR <= '0';
            elsif counter < 700 then       -- dot
               pulseR <= '1';
            elsif counter < 1000 then      -- letter gap
               pulseR <= '0';

            -- O: ---
            elsif counter < 1300 then
               pulseR <= '1';
            elsif counter < 1400 then
```

```
            pulseR <= '0';
        elsif counter < 1700 then
            pulseR <= '1';
        elsif counter < 1800 then
            pulseR <= '0';
        elsif counter < 2100 then
            pulseR <= '1';
        elsif counter < 2400 then
            pulseR <= '0';

        -- C: -.-.
        elsif counter < 2700 then
            pulseR <= '1';
        elsif counter < 2800 then
            pulseR <= '0';
        elsif counter < 2900 then
            pulseR <= '1';
        elsif counter < 3000 then
            pulseR <= '0';
        elsif counter < 3300 then
            pulseR <= '1';
        elsif counter < 3400 then
            pulseR <= '0';
        elsif counter < 3500 then
            pulseR <= '1';
        elsif counter < 3800 then
            pulseR <= '0';

        -- K: -.-
        elsif counter < 4100 then
            pulseR <= '1';
        elsif counter < 4200 then
            pulseR <= '0';
        elsif counter < 4300 then
            pulseR <= '1';
        elsif counter < 4400 then
            pulseR <= '0';
        elsif counter < 4700 then
            pulseR <= '1';
        elsif counter < 5000 then
            pulseR <= '0';

        -- Y: -.--
        elsif counter < 5300 then
            pulseR <= '1';
        elsif counter < 5400 then
            pulseR <= '0';
        elsif counter < 5500 then
            pulseR <= '1';
        elsif counter < 5600 then
            pulseR <= '0';
        elsif counter < 5900 then
            pulseR <= '1';
        elsif counter < 6000 then
            pulseR <= '0';
        elsif counter < 6300 then
            pulseR <= '1';
        else pulseR <= '0'; end if;

        -- BALBOA
-- B: -...
if counter < 300 then
    pulseB <= '1'; -- dash
elsif counter < 400 then
    pulseB <= '0'; -- space
elsif counter < 500 then
    pulseB <= '1'; -- dot
elsif counter < 600 then
    pulseB <= '0';
elsif counter < 700 then
    pulseB <= '1'; -- dot
```

```
elsif counter < 800 then
   pulseB <= '0';
elsif counter < 900 then
   pulseB <= '1'; -- dot
elsif counter < 1200 then
   pulseB <= '0'; -- letter gap

-- A: .-
elsif counter < 1300 then
   pulseB <= '1';
elsif counter < 1400 then
   pulseB <= '0';
elsif counter < 1700 then
   pulseB <= '1';
elsif counter < 2000 then
   pulseB <= '0';

-- L: .-..
elsif counter < 2100 then
   pulseB <= '1';
elsif counter < 2200 then
   pulseB <= '0';
elsif counter < 2500 then
   pulseB <= '1';
elsif counter < 2600 then
   pulseB <= '0';
elsif counter < 2700 then
   pulseB <= '1';
elsif counter < 2800 then
   pulseB <= '0';
elsif counter < 2900 then
   pulseB <= '1';
elsif counter < 3200 then
   pulseB <= '0';

-- B again
elsif counter < 3500 then
   pulseB <= '1';
elsif counter < 3600 then
   pulseB <= '0';
elsif counter < 3700 then
   pulseB <= '1';
elsif counter < 3800 then
   pulseB <= '0';
elsif counter < 3900 then
   pulseB <= '1';
elsif counter < 4000 then
   pulseB <= '0';
elsif counter < 4100 then
   pulseB <= '1';
elsif counter < 4400 then
   pulseB <= '0';

-- O: ---
elsif counter < 4700 then
   pulseB <= '1';
elsif counter < 4800 then
   pulseB <= '0';
elsif counter < 5100 then
   pulseB <= '1';
elsif counter < 5200 then
   pulseB <= '0';
elsif counter < 5500 then
   pulseB <= '1';
elsif counter < 5800 then
   pulseB <= '0';

-- A again
elsif counter < 5900 then
   pulseB <= '1';
elsif counter < 6000 then
```

```
   pulseB <= '0';
elsif counter < 6300 then
   pulseB <= '1';
else pulseB <= '0'; end if;

if counter >= 6300 then counter <= 0;
else counter <= counter + 1; end if;

         end if; end if; end process;

pulse_out <= pulseR when mode = '0' else pulseB;

end Behavioral;
```

cons_lab6.xdc

```
## Clock input (100 MHz on Basys3)
set_property PACKAGE_PIN W5 [get_ports {clk_100MHz} ]
set_property IOSTANDARD LVCMOS33 [get_ports {clk_100MHz}

## Reset and Mode switch
set_property PACKAGE_PIN V17 [get_ports {mode}]
                set_property IOSTANDARD LVCMOS33 [get_ports {mode}]
set_property PACKAGE_PIN V16 [get_ports {reset}]
                set_property IOSTANDARD LVCMOS33 [get_ports {reset}]

## Output signal to oscilloscope
set_property PACKAGE_PIN L2 [get_ports {pulse_out}]
set_property IOSTANDARD LVCMOS33 [get_ports {pulse_out}]
```

tb_lab6.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_lab6 is
--  Port ( );
end tb_lab6;

architecture sim of tb_lab6 is

   -- Component declaration
   component lab6
      Port (
         clk_100MHz : in  STD_LOGIC;
         reset      : in  STD_LOGIC;
         pulse_out  : out STD_LOGIC;
         mode       : in  STD_LOGIC
      );
   end component;

   signal clk_tb       : STD_LOGIC := '0';
   signal reset_tb     : STD_LOGIC := '0';
   signal pulse_out_r   : STD_LOGIC;
   signal pulse_out_b   : STD_LOGIC;

   constant CLK_PERIOD : time := 10 ns; -- 100 MHz

begin

   uut_rocky: lab6
      port map (
         clk_100MHz => clk_tb,
         reset      => reset_tb,
         pulse_out  => pulse_out_r,
         mode       => '0'        );

   uut_balboa: lab6
      port map (
```

```
        clk_100MHz => clk_tb,
        reset      => reset_tb,
        pulse_out  => pulse_out_b,
        mode       => '1'        );

  clk_process : process
  begin
    while now < 2 ms loop
        clk_tb <= '0';
        wait for CLK_PERIOD / 2;
        clk_tb <= '1';
        wait for CLK_PERIOD / 2;
    end loop;
    wait;
  end process;

end sim;
```